

DOCUMENTACIÓN DEL PROYECTO

Sistema de Inventario

Aplicación de escritorio en Java Swing con MySQL

Presentan
Hernández Cruz Karla Sofia
Olvera Hernández Estephany
Higinio Reséndiz Sergio Abimael
García Falcon Luis Raúl

Grupo: 3IRD-G1

Fecha: _____

1. Descripción general

Sistema de Inventario es una aplicación de escritorio diseñada para administrar categorías y productos. Su objetivo es centralizar el registro de existencias, precios y clasificación de artículos mediante una interfaz gráfica sencilla. La solución incorpora programación orientada a objetos, persistencia en una base de datos relacional **MySQL**, operaciones CRUD y manejo controlado de errores.

Funciones principales

- Alta, consulta, modificación y eliminación de categorías.
- Alta, consulta, modificación y eliminación de productos.
- Búsqueda de productos por nombre, incluyendo filtrado al presionar Enter.
- Validación de campos obligatorios, precios y existencias no negativas.
- Restricción de integridad referencial: una categoría con productos no puede eliminarse.
- Confirmación explícita antes de eliminar productos o categorías.

2. Arquitectura orientada a objetos

El proyecto se organiza por responsabilidades. Esta separación favorece el encapsulamiento, la reutilización y el mantenimiento del código.

Paquete	Responsabilidad
model	Entidades Categoría y Producto. Sus atributos son privados y se consultan o modifican mediante getters y setters.
dao	Acceso a datos: consultas SQL parametrizadas (PreparedStatement) y operaciones CRUD.
db	Conexión reutilizable a MySQL e inicialización del esquema (base de datos y tablas) al arrancar la aplicación.
service	Reglas de negocio y validaciones antes de persistir datos.
ui	Ventana principal y diálogo de categorías contruidos con Java Swing.
exception	Excepciones específicas DataAccessException y ValidationException.

3. Persistencia y modelo relacional

La base de datos utiliza **MySQL**, por lo que requiere un servidor de base de datos en ejecución (por ejemplo XAMPP, WAMP o MySQL Server) accesible en `localhost:3306`. La clase `DatabaseConnection` concentra la URL JDBC, crea automáticamente la base de datos `inventario` si no existe y prepara sus tablas mediante el conector oficial `mysql-connector-j`. Los DAO usan

PreparedStatement, lo que evita concatenar entradas del usuario en las sentencias SQL.

Modelo de datos

```
CATEGORIAS PRODUCTOS
-----
id (PK) id (PK)
nombre (UNIQUE, NOT NULL) nombre (NOT NULL)
descripcion precio (CHECK >= 0)
existencia (CHECK >= 0)
categoria_id (FK -> categorias.id)
```

La relación es uno a muchos: una categoría puede clasificar varios productos, pero cada producto debe pertenecer a una sola categoría. La llave foránea y la política ON DELETE RESTRICT conservan la integridad de la información.

Script de base de datos

El archivo database/KSEHSL_3IRD-G1.sql crea ambas tablas y carga tres categorías y tres productos de prueba. Puede ejecutarse con MySQL Workbench, phpMyAdmin o el cliente mysql en línea de comandos.

4. Manejo de excepciones

Las operaciones críticas se protegen con try-with-resources, que cierra automáticamente conexiones, sentencias y resultados. Los errores de SQL se encapsulan en DataAccessException con mensajes comprensibles; las reglas de negocio se comunican mediante ValidationException. La interfaz captura estas excepciones y muestra un cuadro de diálogo, en lugar de finalizar la aplicación inesperadamente.

5. Interfaz gráfica y uso

La ventana principal contiene un formulario de producto, un selector de categoría, una tabla de resultados ordenable por columna y controles para guardar, actualizar, eliminar, limpiar y filtrar. El botón Gestionar categorías abre una segunda ventana para administrar el catálogo de categorías, con la misma confirmación de eliminación que la ventana principal.

Proceso de uso

1. Ejecute el proyecto y cree primero al menos una categoría.
2. Capture nombre, precio, existencia y categoría del producto.
3. Presione Guardar para registrar el artículo.
4. Seleccione una fila de la tabla para cargar sus datos y posteriormente Actualizar o Eliminar.
5. Escriba un texto en Buscar y presione Filtrar (o Enter) para consultar por nombre.

6. Instalación y ejecución

El proyecto incluye un archivo Maven que declara el conector JDBC de MySQL (`mysql-connector-j`). Se requiere JDK 21 o superior, Maven 3.9 o superior y un servidor MySQL en ejecución, con el usuario y la contraseña configurados en `DatabaseConnection.java`. Desde la carpeta raíz del proyecto se ejecuta:

```
mvn clean compile exec:java
```

En el primer inicio se crea automáticamente la base de datos `inventario` y sus tablas, siempre que el servidor MySQL ya esté encendido. Si se desea ver datos de ejemplo desde el inicio, se puede ejecutar previamente el script SQL incluido.

7. Control de versiones

El repositorio Git del proyecto ya está inicializado de forma local, con un archivo `.gitignore` que excluye compilados, configuración de IDE y archivos generados. Para completar el requisito de entrega, el equipo debe:

1. Crear un repositorio remoto en GitHub o GitLab.
2. Vincularlo con `git remote add origin <url>`.
3. Subir el historial con `git push -u origin main`.
4. Asegurar que cada integrante aporte commits propios con mensajes descriptivos, por ejemplo: `git commit -m "Agrega CRUD de productos"`.
5. Compartir acceso público o por invitación al repositorio con el docente.

8. Verificación

La totalidad de las clases Java fue compilada y verificada con JDK 24 en modo de compatibilidad Java 21, usando las dependencias declaradas en pom.xml (mysql-connector-j 9.1.0, slf4j-nop 1.7.36). Antes de entregar, se recomienda ejecutar el comando Maven en un equipo con acceso a Maven Central y a un servidor MySQL activo, verificar los CRUD de productos y categorías, y tomar capturas actualizadas de la aplicación y del historial Git para la evidencia final.

9. Evidencias de funcionamiento

Antes de entregar, sustituya esta sección por capturas propias y actuales del equipo. Se sugiere incluir:

- La ventana principal con productos cargados desde MySQL.
- La ventana de gestión de categorías.
- El script SQL ejecutándose en MySQL Workbench, phpMyAdmin o consola.
- El historial de commits del repositorio («git log --oneline» o la vista de commits en GitHub/GitLab, mostrando aportaciones de cada integrante).